Neural Networks

Lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

11 December 2026 · reading time about 70 minutes

Contents

Lesson plan . . . 2
1. Why this lesson exists . . . 2
    1.1 Three problems . . . 3
2. One neuron is one line . . . 3
    2.1 The perceptron, and the unit you already have . . . 3
    2.2 The acceptance test . . . 4
    2.3 What one line is worth . . . 5
3. A hidden layer: several lines and a vote . . . 7
    3.1 The smallest problem that needs one . . . 7
    3.2 A network written down by hand . . . 7
    3.3 Forward propagation . . . 9
    3.4 Letting training find the weights . . . 10
4. How a network learns . . . 12
    4.1 The loop . . . 12
    4.2 The loss . . . 12
    4.3 One gradient step on one unit . . . 13
    4.4 Backpropagation: the blame flows backwards . . . 14
5. A network in Keras, on the digits . . . 15
    5.1 The digits . . . 15
    5.2 A network in three lines . . . 15
    5.3 Softmax: ten scores become ten probabilities . . . 15
    5.4 Training, watched . . . 16
    5.5 39 points against 2.8 . . . 17
6. Training in practice . . . 18
    6.1 The learning rate . . . 19
    6.2 A network that memorises . . . 19
    6.3 Two remedies . . . 20
    6.4 Activations, and six layers that never learn . . . 21
    6.5 Shuffle the pixels: nothing notices . . . 22
Summary . . . 22
    Homework . . . 23
    Notation used in this lesson . . . 23
Further reading . . . 23

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 8 discussed; what a network adds; three problems	Slides 2–5
0:10–0:30	20	One neuron is one line: the perceptron, the sigmoid, the acceptance test	Slides 6–13
0:30–0:50	20	A hidden layer: several lines and a vote	Slides 14–26
0:50–1:15	25	**Notebook 01** — from one line to a hidden layer	Slide 27
1:15–1:27	12	**Break**	Slide 28
1:27–1:42	15	How a network learns: the loop, mini-batches, one step by hand, the blame flowing back	Slides 29–35
1:42–1:57	15	A network in Keras on the digits; softmax; 39 points against 2.8	Slides 36–44
1:57–2:15	18	**Notebook 02** — a network in Keras	Slide 45
2:15–2:35	20	Training in practice: the learning rate, memorising, two remedies, the vanishing gradient, shuffled pixels	Slides 46–56
2:35–2:55	20	**Notebook 03** — training in practice	Slide 57
2:55–3:00	5	What to take away; homework	Slides 58–60
	180	**Total**	**60 slides, 3 notebooks**

---

1. Why this lesson exists

Every classifier in this course so far drew a boundary whose *shape* was fixed before training started: a straight line for logistic regression (lesson 4), boxes with sides parallel to the axes for a tree (lesson 7), a curve of the kind a chosen kernel allows for a support vector machine (lesson 6). Training only chose *where* that shape went.

A neural network learns the shape itself. It is built from one very simple part, the **unit**, repeated and arranged in layers. The layers between the input and the output are called **hidden layers**, because nobody supplies or labels their values. This lesson is about what one unit does, what a few of them do together, and how they are trained. It leaves out what a first course does not need: the matrix form of the training algorithm, and the many tricks that make very large networks train. Those are in the advanced version of this lesson, in `Advanced/09_neural_networks_advanced/`, for anyone curious; nothing there is examined.

1.1 Three problems

Most of the lesson follows **Meridian Instruments**, a fictional maker of optical sensors, whose data is generated by `Notebooks/instrument_data.py` with every rule published, so a model's result can be compared with the truth.

1. **The acceptance test.** Every sensor is measured on two calibration axes and passes when both offsets are *jointly* small. 3,000 sensors.
2. **The two-channel drift.** A sensor drifts on two channels; it can be corrected when both drift the same way, not when they drift opposite ways. 800 sensors.
3. **Handwritten digits.** 1,797 real images of eight by eight pixels, ten classes, shipped with scikit-learn.

The test rig that records each sensor's verdict is itself wrong 3% of the time. A model that reproduced the true rule perfectly would still disagree with 3% of the recorded verdicts, so on the sensors **no accuracy can exceed 0.97**, and every score is read against 0.97, not against 1.

The number to carry out of this lesson is a comparison: **a hidden layer is worth 39 points on the acceptance test and 2.8 on the digits.** Section 5.5 explains why the same idea is worth so much on one problem and so little on the other, and why that difference is the real lesson.

---

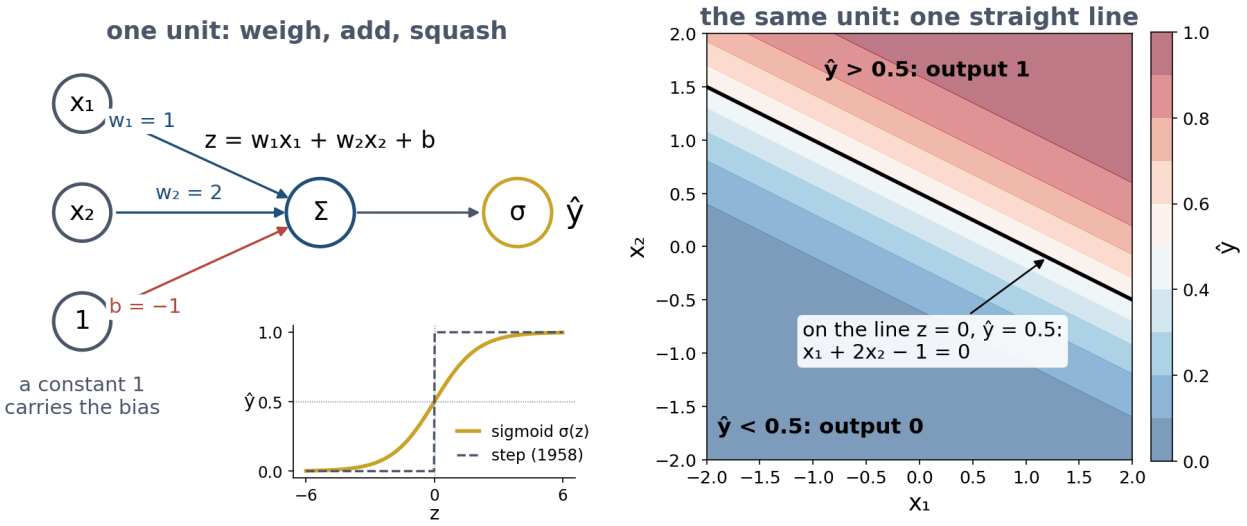
2. One neuron is one line

2.1 The perceptron, and the unit you already have

The **perceptron** (Rosenblatt, 1958) is the ancestor: weights w, a bias b, and the rule "output 1 if $w^\top x + b > 0$, otherwise 0". Replace that hard step with the sigmoid of lesson 4 and you have exactly a logistic regression unit:

$$\hat{y} = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Nothing here is new. What is new is treating this as a *component*, a **unit** or **neuron**, which a network repeats many times. The function applied to the weighted sum, here the sigmoid, is the unit's **activation function**, and its output is the unit's **activation**.



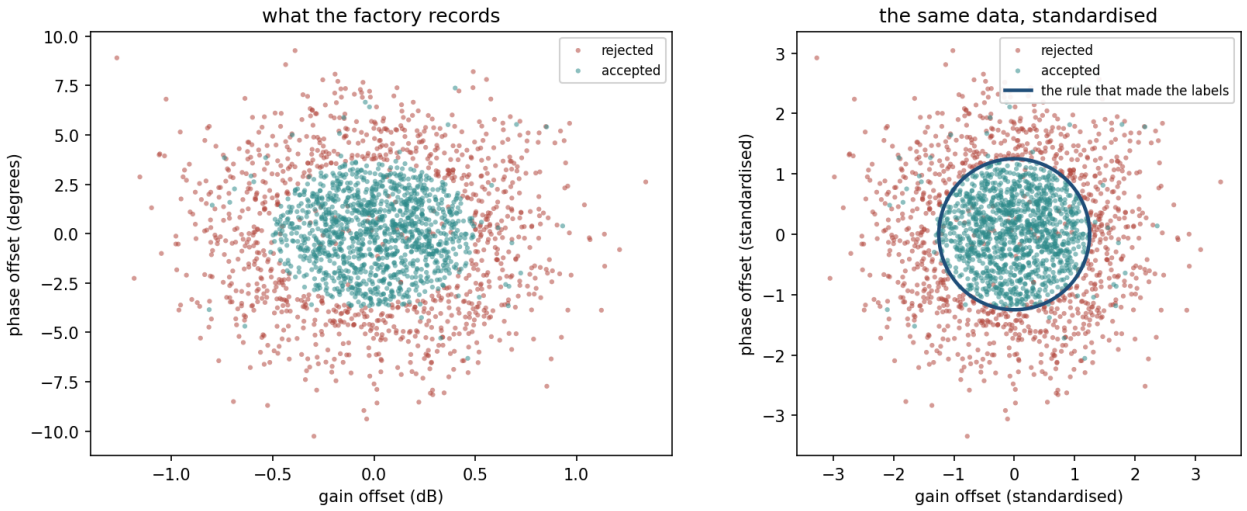
One unit drawn two ways, with $w_1 = 1$, $w_2 = 2$, $b = -1$. Left: the inputs are weighted and added, with a constant 1 carrying the bias, and the sum z is squashed by the sigmoid; the small plot shows the sigmoid against the step the 1958 perceptron used, both switching at $z = 0$. Right: the same unit as a map of the input plane. Look at the black line $x_1 + 2x_2 - 1 = 0$: on it $z = 0$ and $\hat{y} = 0.5$, and the unit's verdict is which side of it a point lies on.

Worked example. With those weights, notebook 01 computes four points by hand. The point $(1, 1)$ gives $z = 1 \cdot 1 + 2 \cdot 1 - 1 = 2$ and $\hat{y} = \sigma(2) = 0.8808$: class 1. The point $(0, 0)$ gives $z = -1$ and $\hat{y} = 0.2689$: class 0. The point $(1, 0)$ gives $z = 0$ and $\hat{y} = 0.5000$ exactly: it lies on the line, and the unit is undecided.

So the set of points a unit is undecided about, $w^\top x + b = 0$, is a straight line in two dimensions, a plane in three, a flat "hyperplane" in general. **A neuron is a line.** Everything one unit can express is "which side of this line are you on, and how far".

2.2 The acceptance test

Each sensor has a gain offset (in decibels) and a phase offset (in degrees). Notebook 01 splits the 3,000 sensors into 2,250 for training and 750 for testing, then standardises each axis with a scaler fitted on the training part.

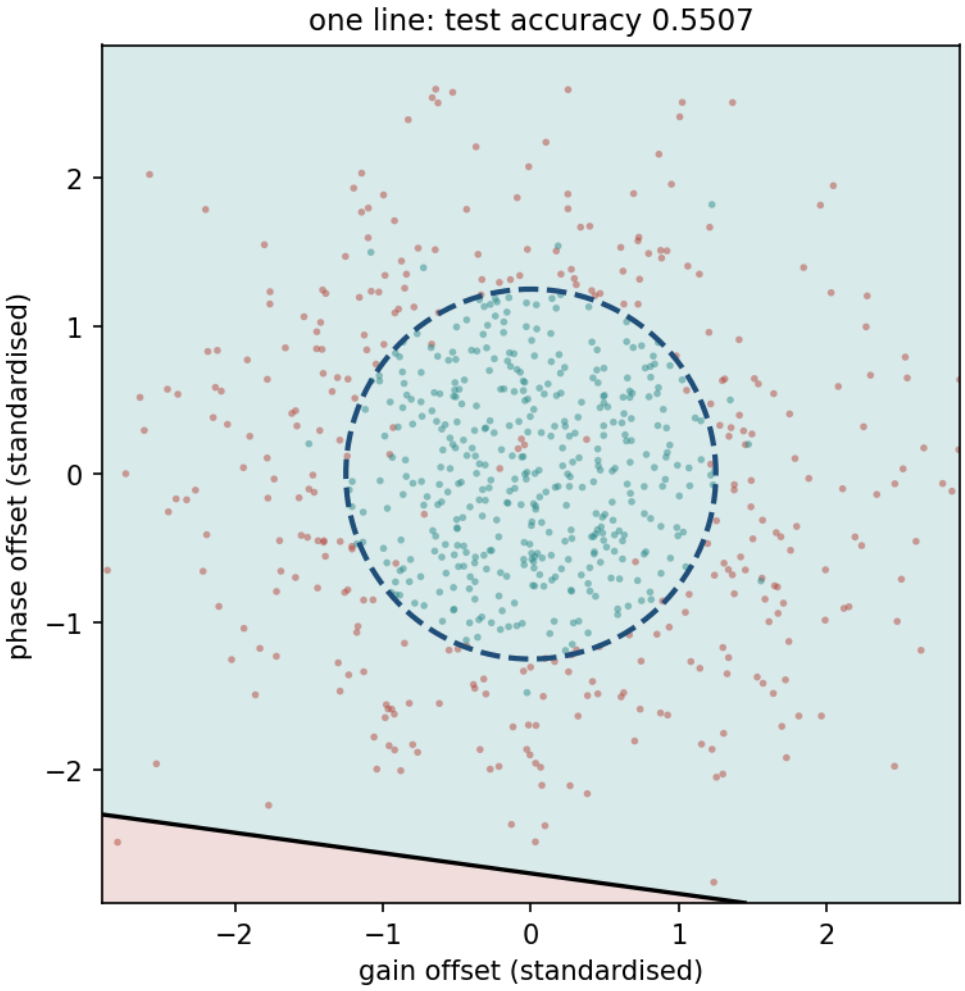


Left: the raw measurements. Right: the same points after each axis is divided by its own production spread, with the rule that made the labels. Look at the right panel: a sensor passes inside a circle, and the few wrong-coloured points along the circle are the rig's 3% of mistaken verdicts.

The rule that made the labels divides each offset by its tolerance and adds the squares: a sensor passes when $(\text{gain}/0.50)^2 + (\text{phase}/3.75)^2 < 1$. The circle is not a teaching choice. Both tolerances are exactly 1.25 times the production spread of their axis (0.50 dB against 0.40, 3.75 degrees against 3.00), so after standardising, a sensor passes when it lies within 1.25 of the centre: a circle of radius 1.25. (The scaler uses the training sensors' own mean and spread, which are close to 0 and to 0.40 and 3.00, so the circle is very nearly exact.)

2.3 What one line is worth

	accuracy
always predict "accepted", on the 750 test sensors	0.5480
logistic regression (one line), on the 750 test sensors	**0.5507**
the rig's ceiling	0.9700



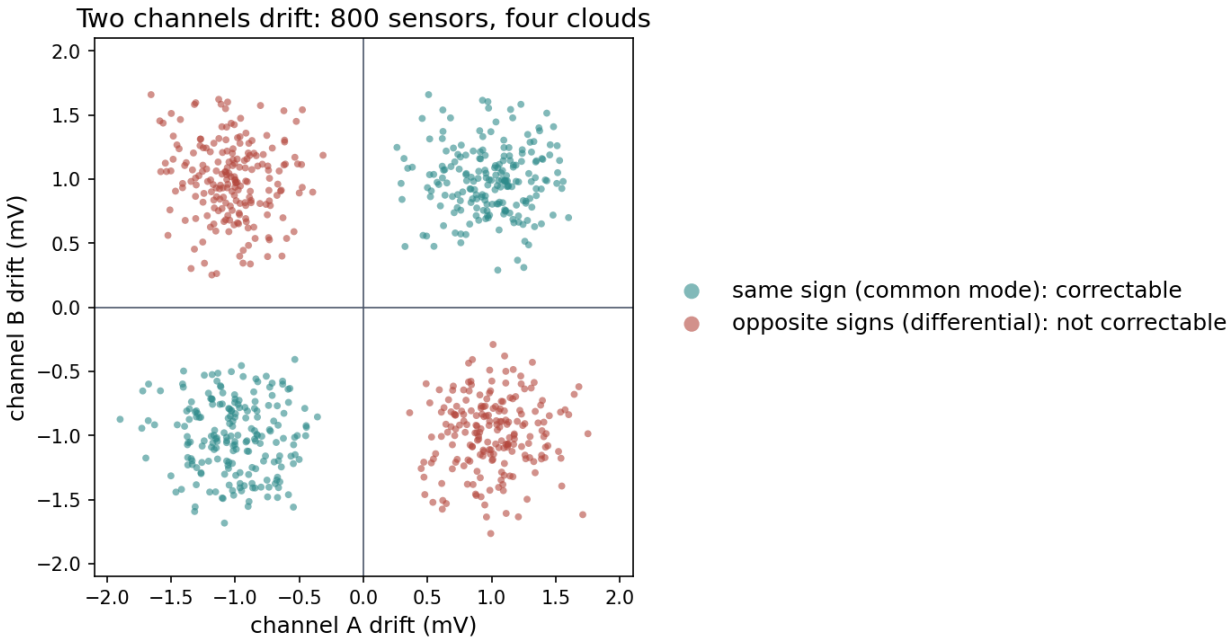
The fitted logistic regression on the test sensors, with the true circle dashed. Look at where the line went: almost off the picture, so that nearly everything is called "accepted".

The predictable mistake, and why it is reasonable. A result of 0.5507, barely above always saying "accepted", looks like a bug: an optimiser that gave up. Usually that diagnosis would be right. Here it is wrong. The fitted coefficients are $(0.0096, 0.0701)$ with intercept 0.1892, almost exactly a constant model, and that is the correct answer. The accept region is a disc centred on the origin: for every accepted sensor there is, on average, a matching one on the opposite side. Whatever a line gains by tilting one way, it gives back on the other side, so the best it can do is barely tilt at all. **No straight line can enclose a region.**

---

3. A hidden layer: several lines and a vote

3.1 The smallest problem that needs one



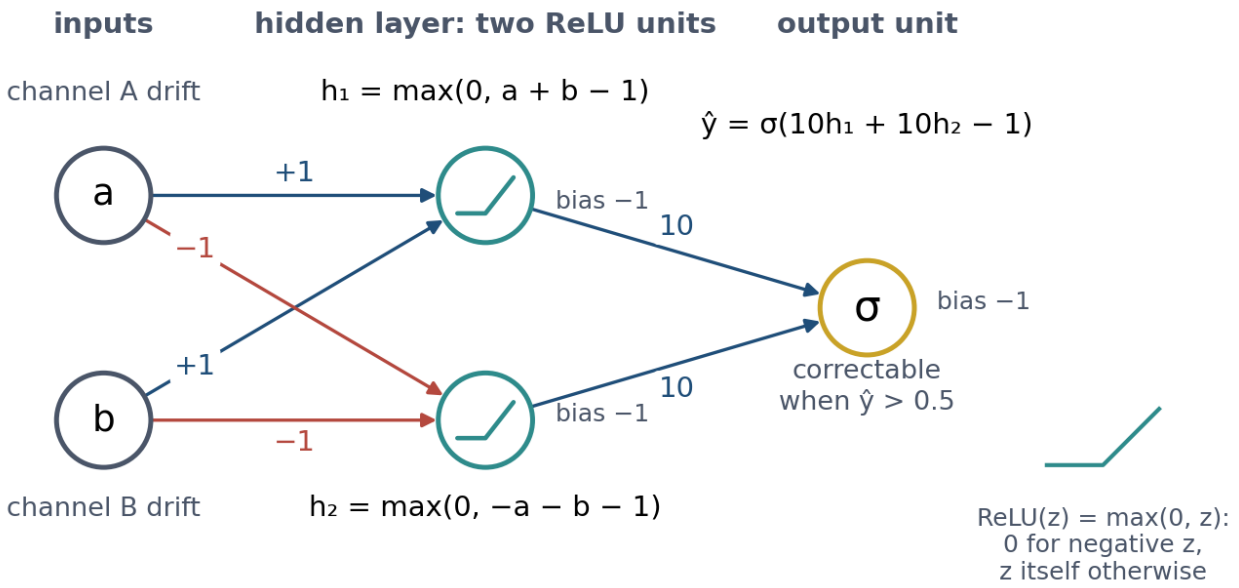
800 sensors with two drifting channels, in four clouds. Look at the colours: the two clouds where both channels drift the same way (correctable) sit on opposite corners.

The rule follows from the electronics: one global correction removes a drift that is the same on both channels, but not one that is opposite. In logic this is the exclusive-or (XOR) of the two signs. Logistic regression scores exactly **0.5000**: a line is worth nothing at all.

3.2 A network written down by hand

Both correctable clouds have $a + b$ near $+2$ or -2; both uncorrectable ones have $a + b$ near 0. So the rule is "$|a + b|$ is large", and that needs **two** boundaries, $a + b > 1$ and $a + b < -1$, which one line cannot provide. Two hidden units can.

Each hidden unit here uses the **rectified linear unit (ReLU)**, $\mathrm{ReLU}(z) = \max(0, z)$: zero for negative z, z itself otherwise.

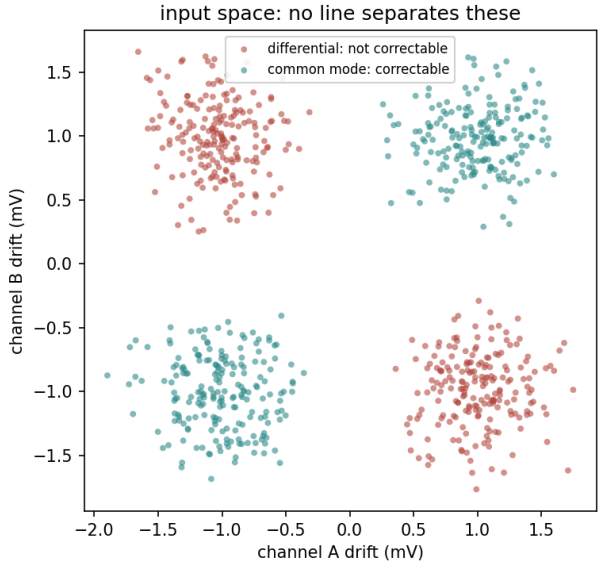


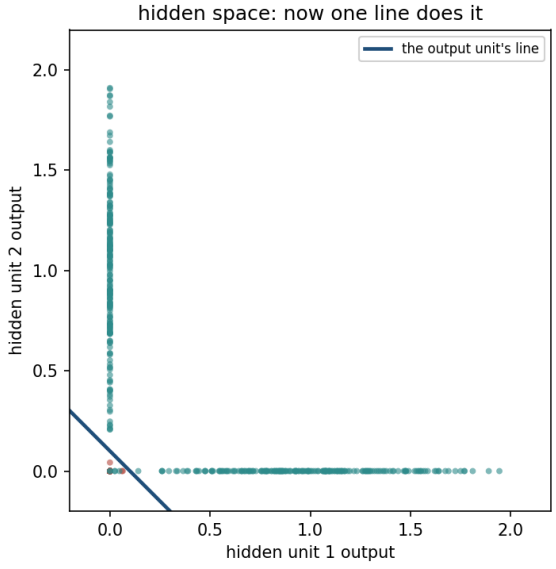
The hand-built network. Hidden unit 1 computes $\max(0, a+b-1)$*, hidden unit 2* $\max(0, -a-b-1)$*; the output unit adds them, ten times each, minus 1, and applies the sigmoid. Follow one cloud through it in the table below.*

Worked example: the four cloud centres.

centre (a, b)	h_1	h_2	$10h_1+10h_2-1$	$\hat{y}$	class
$(+1, +1)$	1	0	9	0.9999	1, correctable
$(-1, -1)$	0	1	9	0.9999	1, correctable
$(+1, -1)$	0	0	-1	0.2689	0
$(-1, +1)$	0	0	-1	0.2689	0

On all 800 sensors, with nothing trained, this network scores **0.9938**.





Left: the sensors as measured, which no line separates. Right: the same sensors as the two hidden units see them. Look at where the clouds went: the correctable ones to $(1, 0)$ *and* $(0, 1)$*, the others together at the origin, and now one line separates them.*

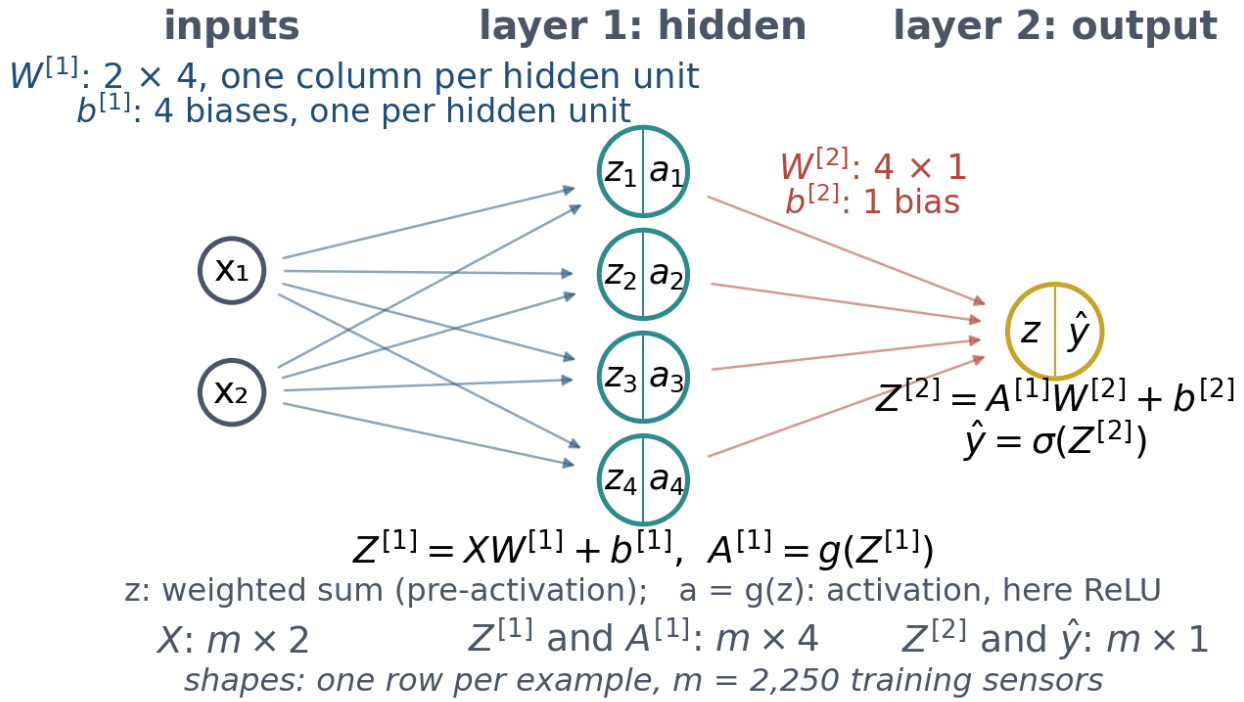
That picture is the central idea of the lesson. **The hidden layer did not classify anything. It moved the data** into new coordinates where the output unit's single line is enough. Learning a network means learning such coordinates, and that is called **representation learning**.

3.3 Forward propagation

Computing a network's output from its input is the **forward pass**. For one hidden layer of H units, with the m examples stacked as the rows of X:

$$Z^{[1]} = XW^{[1]} + b^{[1]}, \qquad A^{[1]} = g(Z^{[1]}), \qquad Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}, \qquad \hat{y} = \sigma(Z^{[2]})$$

The superscript in square brackets is the layer, g is the hidden activation (here ReLU), Z holds weighted sums and A activations.



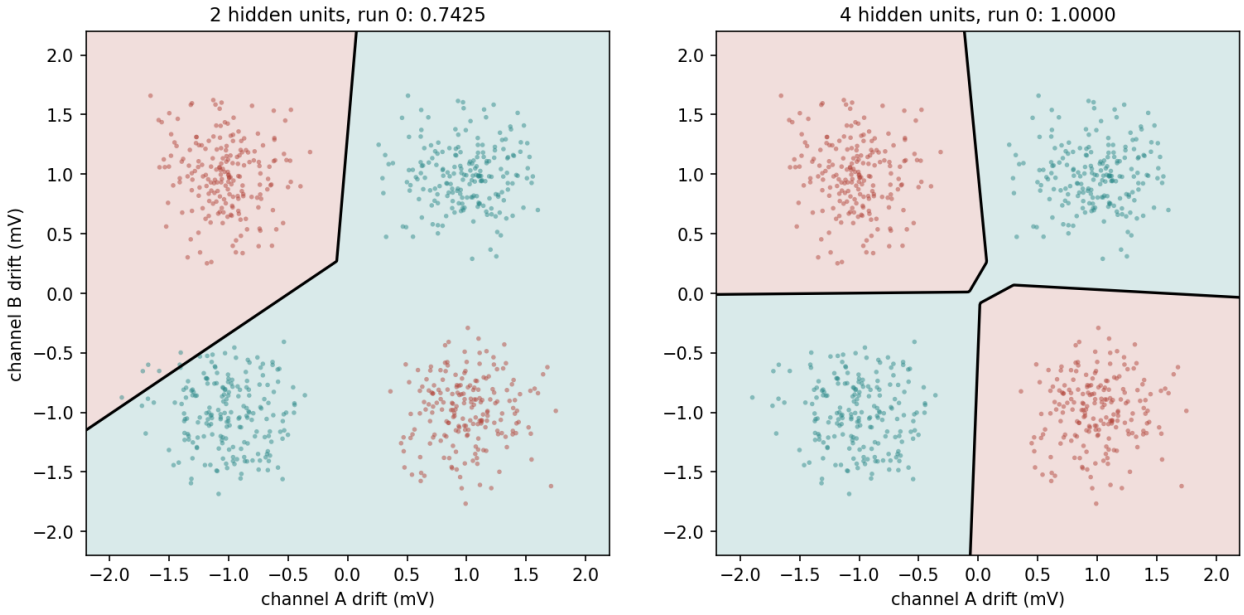
A network with two inputs, four hidden units and one output, with every symbol in its place. Weights live on the edges: $W^{[1]}$ *is* 2×4*, one column per hidden unit. Each hidden node holds two numbers, its weighted sum* z *and its activation* $a = g(z)$*. Look at the bottom row: stacking all* m *examples as rows turns those per-node numbers into the matrices* $Z^{[1]}$*,* $A^{[1]}$ *and* $Z^{[2]}$*.*

Notebook 01 writes this pass for the hand-built network in two lines of NumPy: `hidden = relu(D @ W1 + b1)` and `y_hat = sigmoid(hidden @ W2 + b2)`.

3.4 Letting training find the weights

Nobody writes weights by hand in practice. scikit-learn's `MLPClassifier` (multilayer perceptron) trains a network with one hidden layer of a chosen size. Training starts from small random weights, and the starting point, set by `random_state`, matters: notebook 01 trains twenty times per size on the drift problem, with all 800 sensors used for training and scoring.

hidden units	runs reaching 0.99	median accuracy
2	6 of 20	0.7381
4	17 of 20	1.0000
8	20 of 20	1.0000



The first run at each size. Look at the left panel: with two units, training walled off three of the four clouds and stopped there, at 0.74.

The predictable mistake. It is natural to think that if two units *can* solve a problem, training two units will. Section 3.2 shows two units can; the table shows training finds that solution in only 6 runs of 20, while the rest stop at three clouds of four. A wider layer than the problem strictly needs gives training more chances to start near a good set of lines, which is why networks used in practice are wider than their task requires.

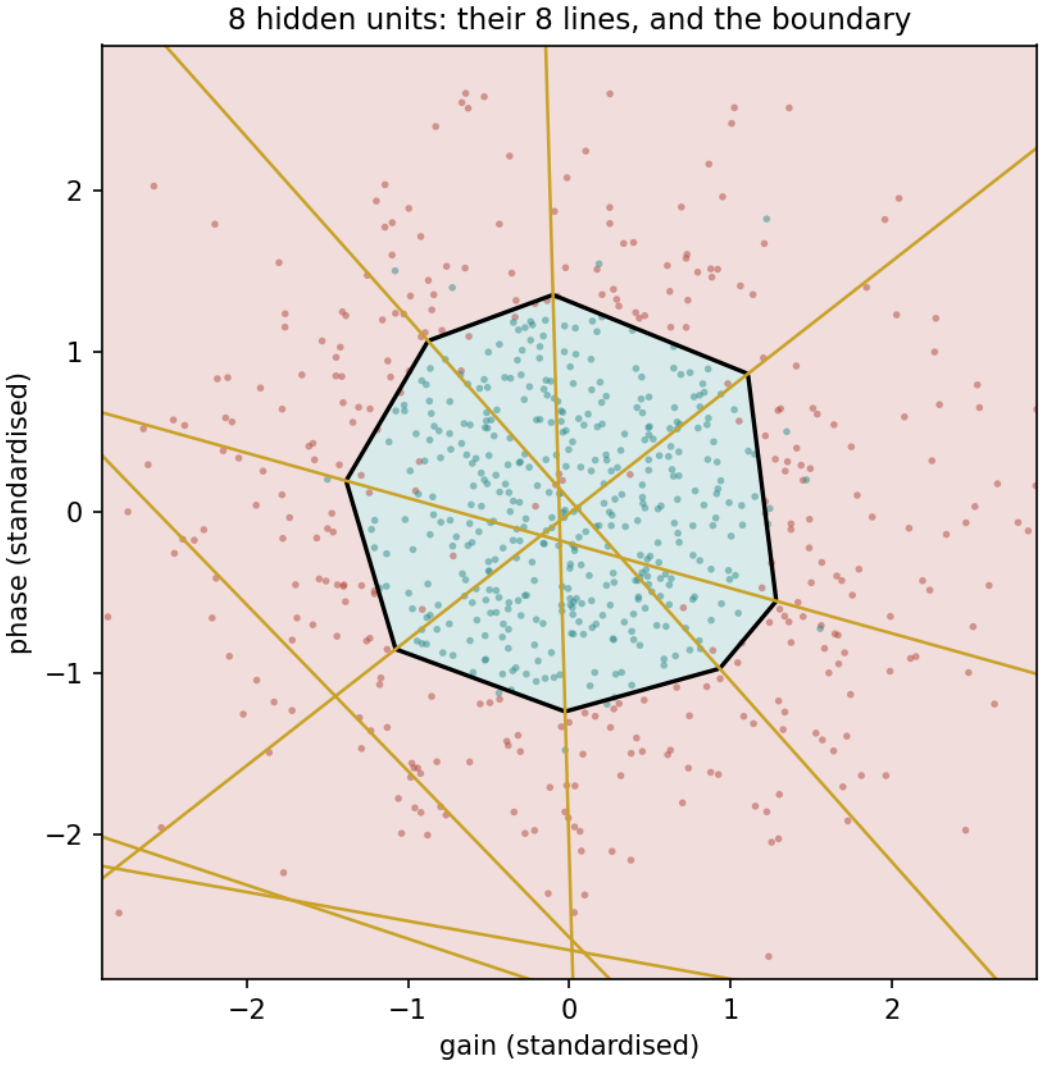
Back to the acceptance test, scored on the 750 test sensors:

hidden units	mean of 5 runs	worst	best
1	0.6048	0.5480	0.6493
2	0.7392	0.7240	0.7693
3	0.8120	0.7227	0.9160
4	0.8821	0.7227	0.9507
8	**0.9445**	0.9387	0.9493
16	0.9549	0.9520	0.9600

hidden units	mean of 5 runs	worst	best



The first run at 1, 2, 4 and 8 hidden units, with the true circle dashed. Look at the shape of the boundary: one unit gives one line, two give a strip, four a four-sided fence, eight an eight-sided one close to the circle.



The eight hidden units of that network, each drawn as its own line from its column of the first weight matrix (`coefs_[0]`), with the network's boundary in black. Look at how the output unit's vote over which side of each line a point lies on encloses a region that no single line could.

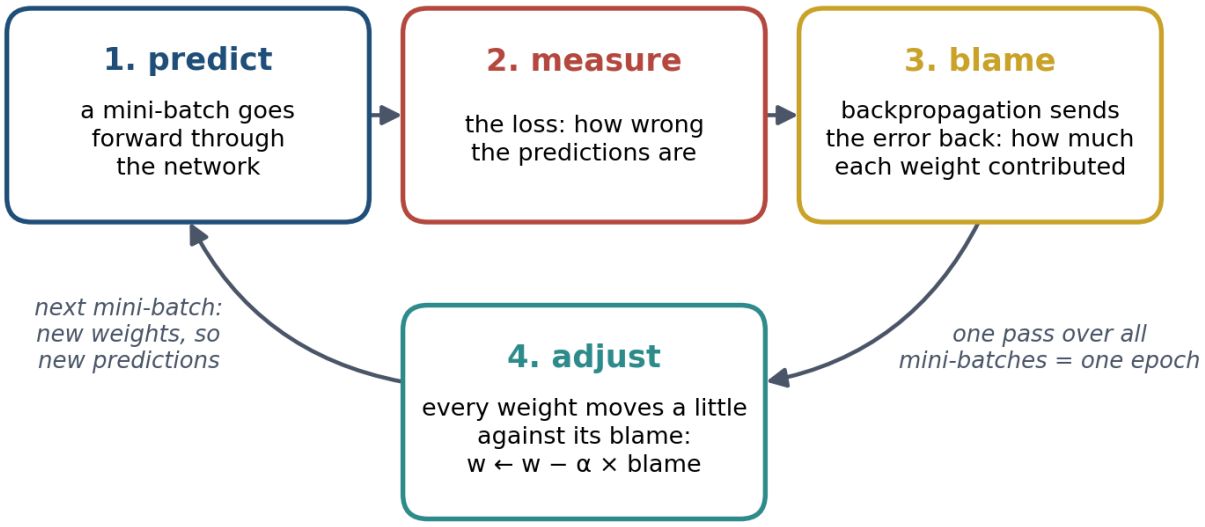
With eight units the mean, **0.9445**, is within three points of the ceiling of 0.97. From one line, 0.5507, that is **39 points**.

---

4. How a network learns

4.1 The loop

Training repeats four steps, many times:



Training as a loop. Look at the arrow back to step 1: after the weights move, every prediction, every error and every blame has to be computed again; only the weights carry over from one round to the next.

One word in the figure needs a definition. A weight's **blame** is how much of the error it is responsible for: precisely, its gradient, the rate at which the loss changes when that weight changes. Section 4.3 computes it for one unit.

Two more words name how the data is fed in. A **mini-batch** is a small random group of training examples, 32 in notebook 02; each round of the loop uses one mini-batch, which is much cheaper than using all the data and still points roughly the right way on average. An **epoch** is one pass through all the mini-batches, so through every training example once. Notebook 02 keeps 1,077 of the 1,797 digits for training (section 5.1 gives the full split); in mini-batches of 32 that is $1{,}077/32 = 33.7$, so 34 mini-batches per epoch, the last one smaller, and 60 epochs make $60 \times 34 = 2{,}040$ rounds of the loop.

4.2 The loss

The loss measures how wrong the predictions are, and it is the one lesson 4 used for logistic regression, the **cross-entropy** (log-loss). For one example with true label $y \in \{0, 1\}$ and prediction $\hat{y}$:

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

It is small when $\hat{y}$ is close to y and grows without limit as a confident prediction turns out wrong. For ten classes it becomes $-\log$(probability given to the true class).

4.3 One gradient step on one unit

Each weight moves against its **gradient**, the slope of the loss with respect to that weight, scaled by the learning rate α (lesson 3):

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w}$$

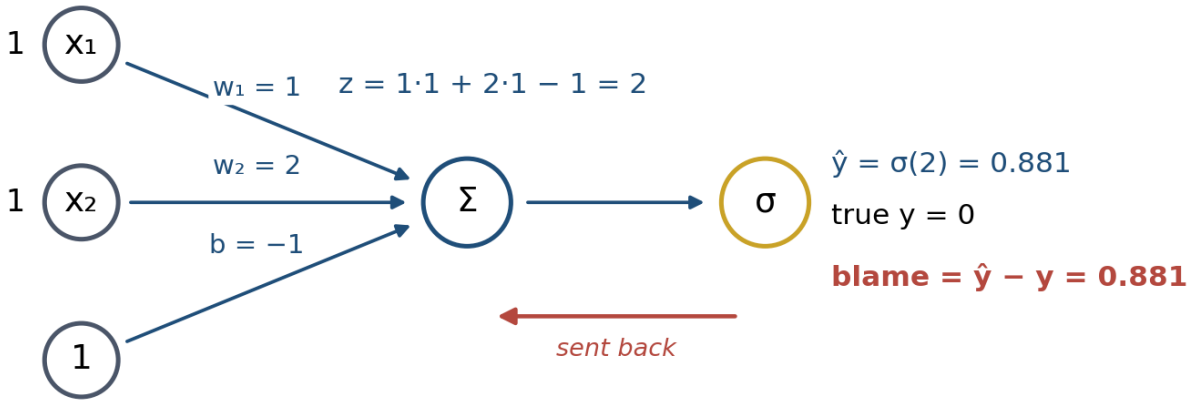
For one sigmoid unit with the cross-entropy loss, the chain rule gives a strikingly simple result. Write $z = w_1 x_1 + w_2 x_2 + b$ and $\hat{y} = \sigma(z)$. Then

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \right) \hat{y}(1 - \hat{y}) = \hat{y} - y$$

using the sigmoid's own derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Since $\partial z / \partial w_j = x_j$ and $\partial z / \partial b = 1$:

$$\frac{\partial L}{\partial w_j} = (\hat{y} - y)\, x_j, \qquad \frac{\partial L}{\partial b} = \hat{y} - y$$

In words: **the blame for the output is the prediction minus the truth, and each weight's blame is that times the input it carried.** It is lesson 4's logistic regression gradient, for one example.



each weight's blame = 0.881 × its input; step with α = 0.5:

w₁ → 0.560, w₂ → 1.560, b → −1.440

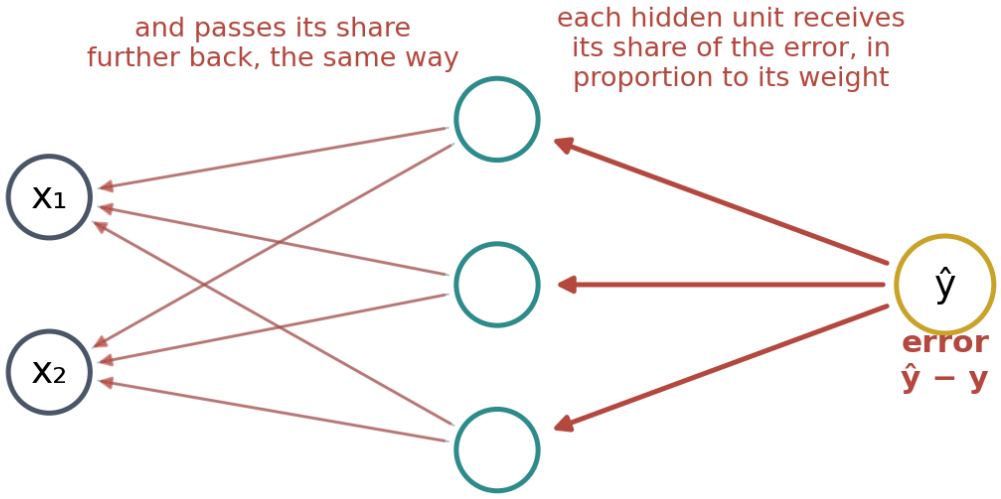
new prediction: ŷ = σ(0.679) = 0.663, closer to the true 0

One step, every number shown. Forward in blue, blame in rust. Look at the last line: one step moves the prediction from 0.881 to 0.663, towards the true label 0.

Worked example. The unit of section 2.1, $w = (1, 2)$, $b = -1$, sees $x = (1, 1)$ whose true label is $y = 0$. Forward: $z = 2$, $\hat{y} = \sigma(2) = 0.881$, loss $L = -\log(1 - 0.881) = 2.127$. Blame: $\hat{y} - y = 0.881$, and since both inputs are 1, every weight's blame is 0.881. A step with $\alpha = 0.5$ moves each by $0.5 \times 0.881 = 0.440$: $w_1 \to 0.5596$, $w_2 \to 1.5596$, $b \to -1.4404$ (the figure rounds them to three decimals). The new z is $0.5596 + 1.5596 - 1.4404 = 0.679$ and the new prediction $\sigma(0.679) = 0.663$: closer to 0, with loss down to 1.089.

4.4 Backpropagation: the blame flows backwards

In a network, a hidden unit's weights do not touch the loss directly; they affect it through the output. **Backpropagation** applies the same chain rule one layer at a time, from the output back to the input: the output's error is shared out among the hidden units in proportion to the weights connecting them, each hidden unit scales its share by the slope of its own activation, and passes it further back the same way.



one backward sweep gives every weight its blame, layer by layer, right to left

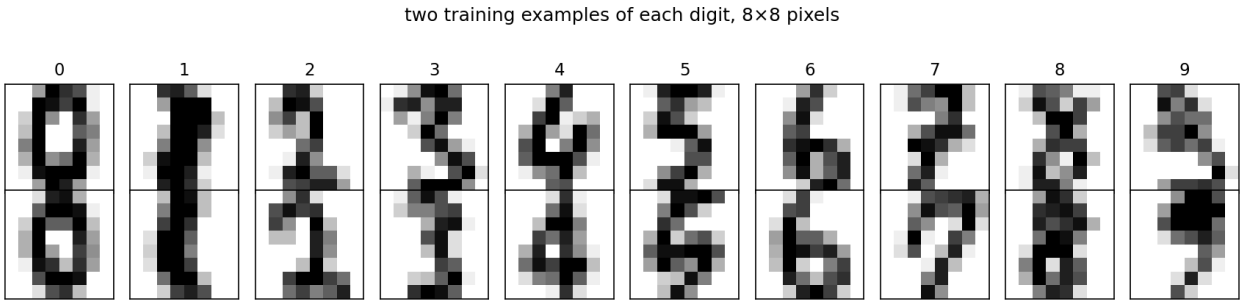
The backward sweep. Look at the direction of the arrows: the error starts at the output and travels back along the same connections the forward pass used, so one sweep gives every weight its blame.

Two facts make it practical. One backward sweep costs about as much arithmetic as one forward pass, so a gradient for every weight costs roughly one more prediction. And the libraries do it for you: `MLPClassifier` and Keras both run backpropagation inside `fit`.

---

5. A network in Keras, on the digits

5.1 The digits



Two training examples of each digit, eight by eight pixels. Look at how coarse they are: some are hard for a person to read.

The 1,797 images are split three ways, as lesson 5 asks: 1,077 to train, 360 to validate (to compare designs), 360 to test (touched once, at the end). Each pixel is divided by 16 so it lies between 0 and 1.

5.2 A network in three lines

```
model = keras.Sequential([
    keras.Input(shape=(64,)),                  # 64 pixels in
    layers.Dense(32, activation="relu"),       # hidden layer: 32 units
    layers.Dense(10, activation="softmax"),    # output: 10 probabilities
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
history = model.fit(X_train, y_train, validation_data=(X_val, y_val),
                    epochs=60, batch_size=32)
```

`Dense` is a layer in which every unit sees every input. `compile` chooses the loss, cross-entropy for integer class labels, and the optimiser, **Adam**, a refined form of gradient descent that adapts the step for each weight. `fit` runs the loop of section 4.1.

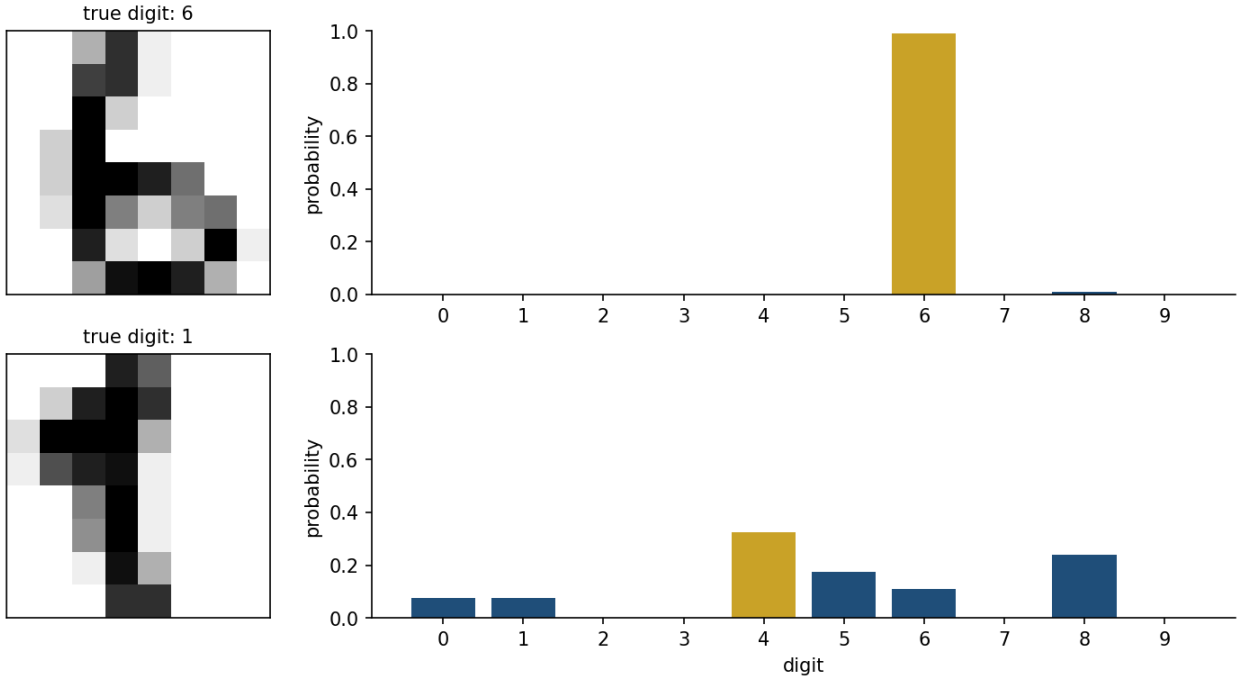
Worked example: counting the weights. The hidden layer has one weight per pixel and unit plus one bias per unit, $64 \times 32 + 32 = 2{,}080$. The output layer has $32 \times 10 + 10 = 330$. Total **2,410**, the number Keras prints.

5.3 Softmax: ten scores become ten probabilities

The output layer computes ten numbers per image, one per digit. **Softmax** makes them positive and makes them add up to 1:

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

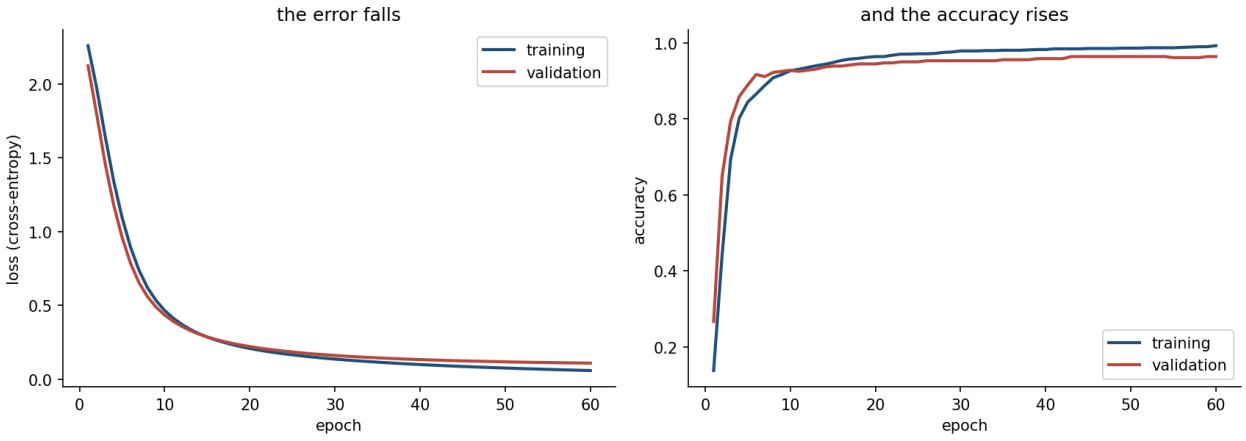
The prediction is the digit with the largest probability. With two classes, softmax reduces to the sigmoid.



Two validation images and the ten probabilities the network gives each. Look at the difference: the 6 above gets 0.991 on one digit; the 1 below, which the network gets wrong, has no digit above a third.

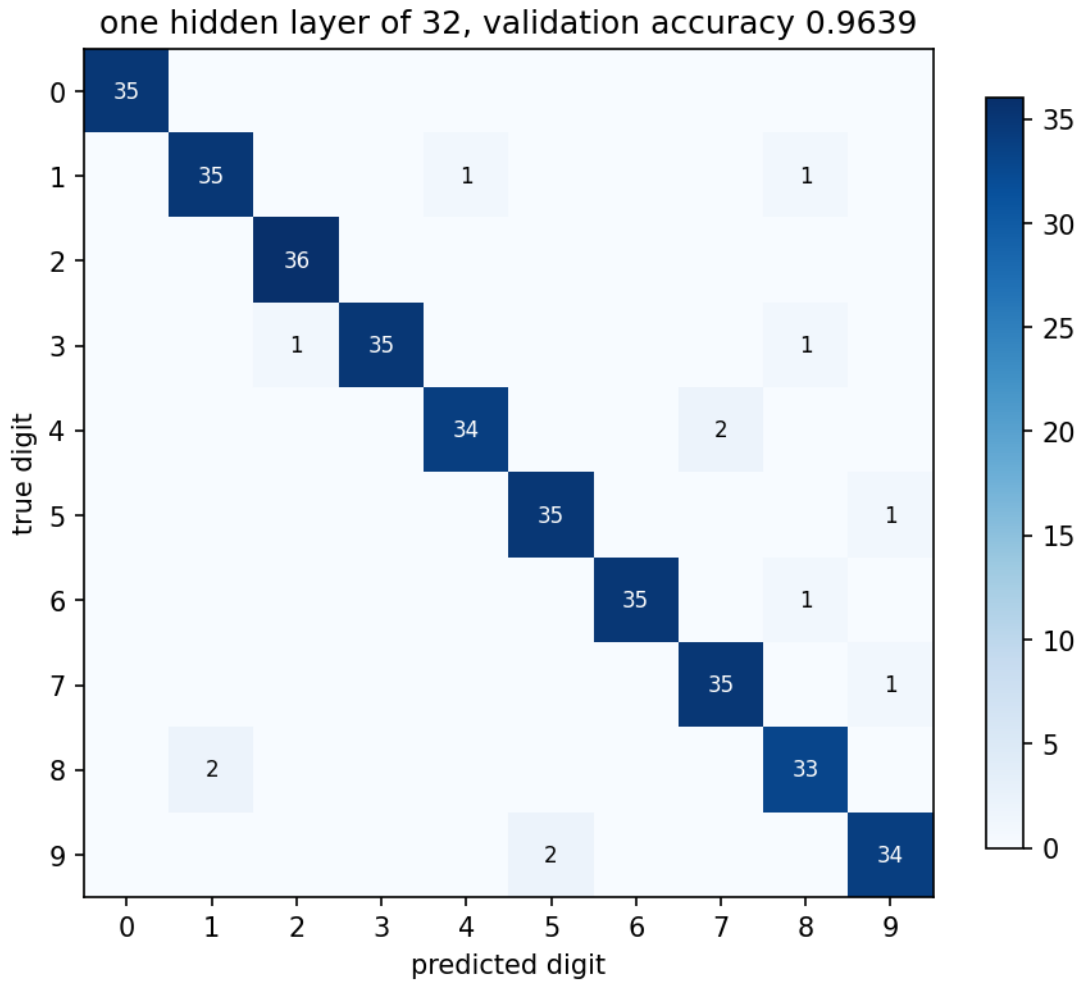
The predictable mistake. It is tempting to read the largest probability as how likely the prediction is to be right. Softmax says how *sure* the network is, not how *right*: notebook 02 also finds a 4 called a 7 with probability 0.920.

5.4 Training, watched



Loss and accuracy after every epoch, on the training and validation images. Look at the first ten epochs: almost all the learning happens there.

After the first epoch the network scores 0.2667 on the validation images; after 60 it scores 0.9639, getting 13 of the 360 wrong, and 0.9926 on the images it trained on.



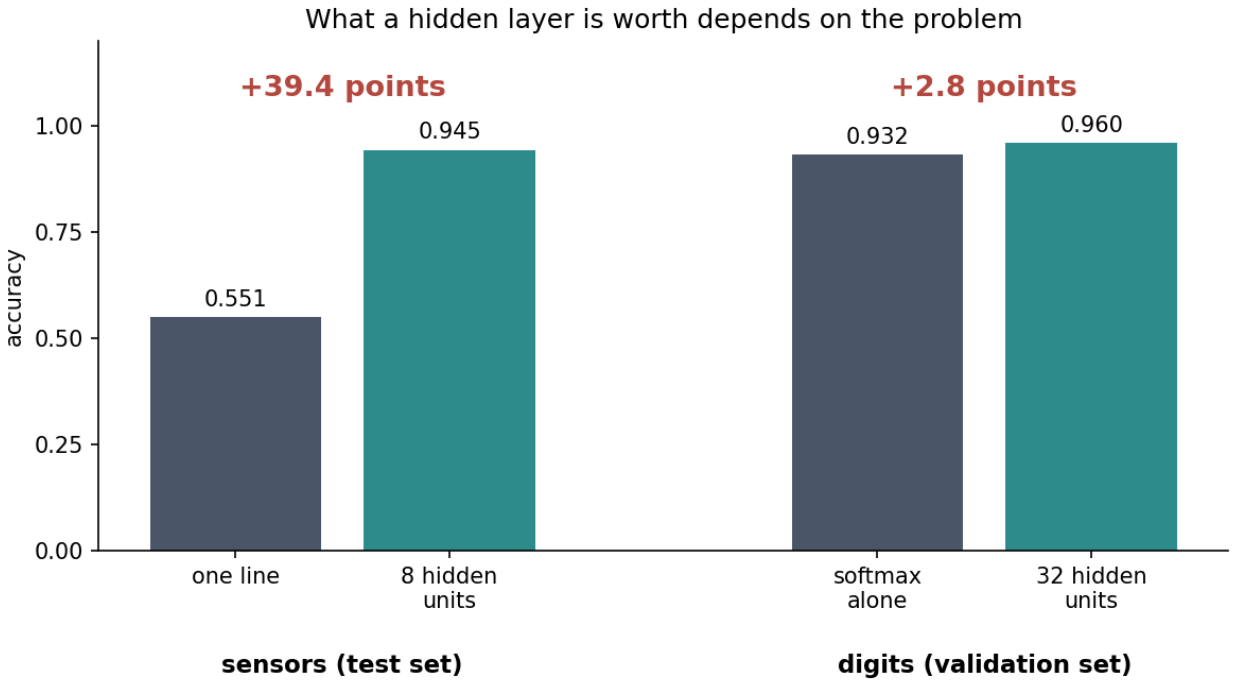
Which digits are confused with which, on the validation images. Look off the diagonal: the mistakes are few and scattered, a 4 taken for a 7, a 9 for a 5.

5.5 39 points against 2.8

Remove the hidden layer and the network is softmax alone: one weighted sum of the pixels per digit, which is multiclass logistic regression. Three runs each, on the validation images:

architecture	parameters	validation accuracy	sd over 3 runs
softmax alone, no hidden layer	650	0.9324	0.0035
one hidden layer of 32	2,410	**0.9602**	0.0052
one hidden layer of 64	4,810	0.9611	0.0045

(The single run of section 5.4 scored 0.9639; this table averages three.)



What a hidden layer is worth on each problem. Look at the two gaps: 39.4 points on the sensors, 2.8 on the digits.

Is 2.8 a property of the digits? Notebook 02 draws a different train/validation split and three other starting points: the gap comes out at 4.7 points. The exact figure moves with the split; its size does not. **A few points on the digits, about 39 on the sensors**: the same idea is worth roughly ten times more on one problem than on the other, and the reason is the shape of each problem. On the sensors the answer depends on the two offsets **together**: a sensor passes when the pair is near the centre, and no weighted sum of the two can say "near the centre". On the digits each pixel pushes the answer more or less **on its own**: ink here suggests a 0, ink there a 1, and a weighted sum of pixels already gets most of the way.

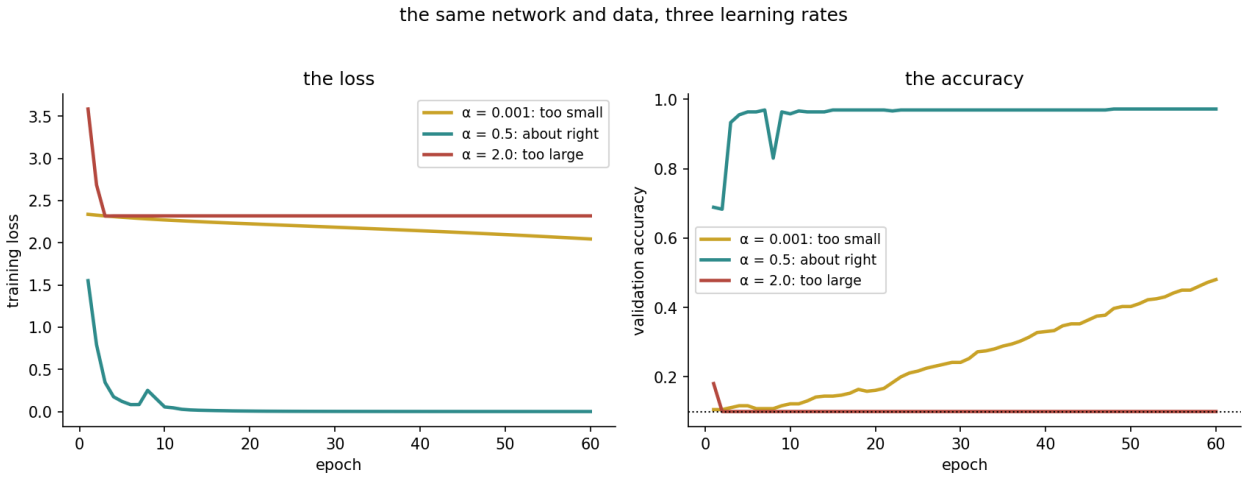
The predictable mistake. Having seen a hidden layer take the sensors from 0.55 to 0.94, it is natural to add one by reflex. The instinct is sound when inputs act in combination; on data where they act independently it buys little and costs parameters, here almost four times as many (2,410 against 650) for 2.8 points. **Fit the linear model first: the size of the gap tells you whether a network was needed.**

---

6. Training in practice

Notebook 03 takes the digits of section 5 and shows the things that most often go wrong when a network is trained: a learning rate set badly, a network that memorises, a deep stack of sigmoids that never learns, and a blindness no training can fix.

6.1 The learning rate



The same network, two hidden layers of 64 trained with plain stochastic gradient descent (SGD) on the 1,077 training digits for 60 epochs, at three learning rates, one run each, scored on the 360 validation digits. Look at the right panel: at $\alpha = 0.001$ *accuracy is still climbing when training stops, at 0.5 it settles high, at 2.0 it never leaves 0.1.*

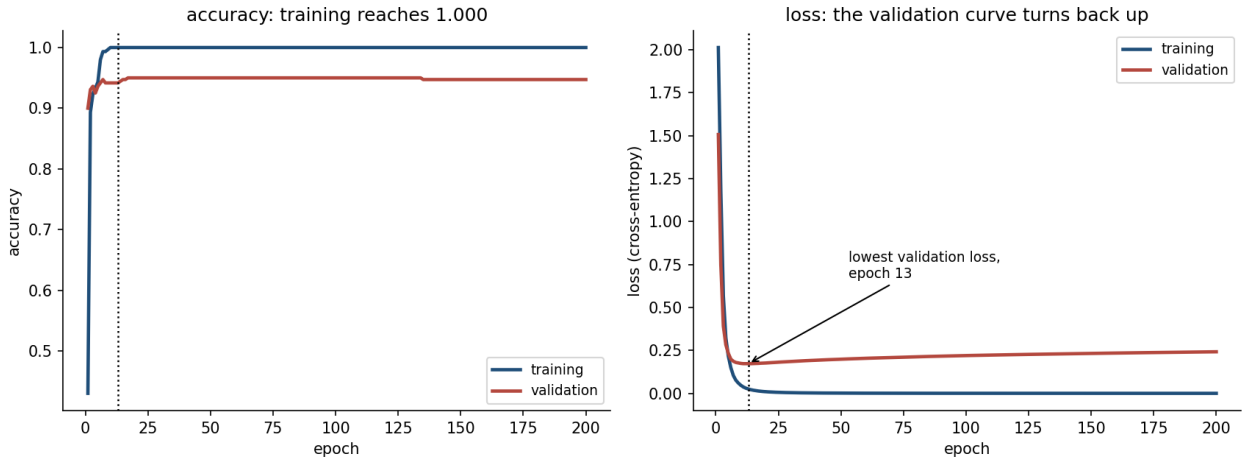
learning rate α	final training loss	validation accuracy
0.001	2.0456	0.4806
0.5	0.0007	**0.9722**
2.0	2.3195	0.1000

Too large, and every step overshoots, so the loss never comes down: 0.1000 is what guessing scores on ten classes. Too small, and the network is still learning when the epochs run out.

The predictable mistake. A rate too small gives low accuracy on both the training and the validation data, which is exactly what a network *too small* looks like, and the natural response is to make the network bigger. That reasoning is sound in general: low training accuracy is the usual sign of a model too simple. Here it is the wrong fix. **Try learning rates a factor of ten apart before changing the network.**

6.2 A network that memorises

Notebook 03 trains two hidden layers of 512 units on only 300 training digits: 301,066 weights and biases, a thousand per example.



200 epochs. Left: training accuracy reaches 1.000 within a few epochs and stays there; validation accuracy stays roughly level. Right: look at the validation loss: lowest at epoch 13, then climbing for the rest of training.

The network has enough freedom to store the 300 answers outright, and it does. The validation accuracy, 0.9472 after 200 epochs, hides the damage; the validation *loss*, 1.4 times its lowest value by the end, shows it: the network is growing more confident about the validation images it gets wrong.

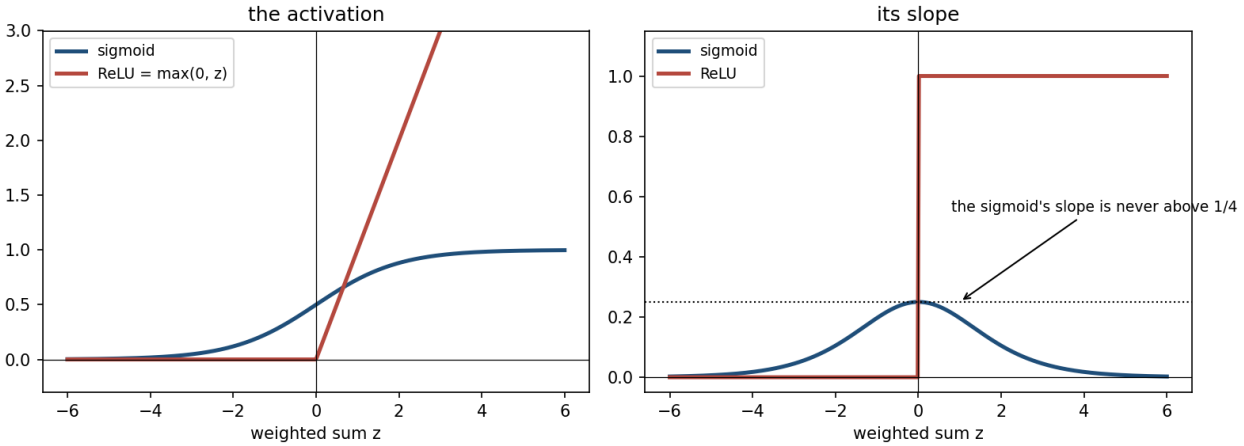
6.3 Two remedies

Early stopping watches the validation loss and stops when it has not improved for a set number of epochs (25 here), keeping the weights from the best epoch. **Dropout** switches off a random 40% of the hidden units on every training mini-batch, so no unit can rely on any particular other one. Three runs each, scored on the test images, which nothing here was chosen against:

method	test accuracy	sd over 3 runs	epochs run
200 epochs, no remedy	0.9426	0.0013	200
early stopping	0.9463	0.0035	39
early stopping + dropout 0.4	0.9481	0.0035	50.3

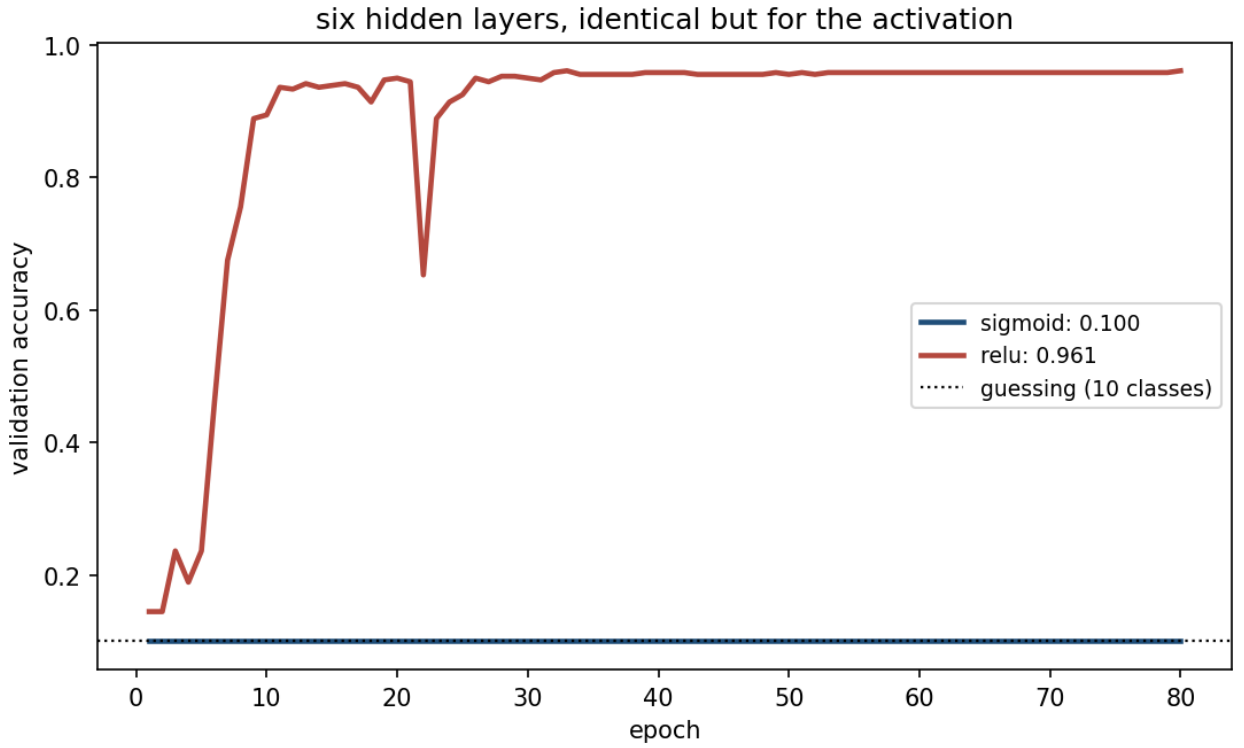
Read the spread before the means, as lesson 5 taught. The best and worst rows differ by 0.0055, under two standard deviations of the runs: enough to say the remedies did not hurt, not enough to rank them. What early stopping certainly bought is time, 39 epochs instead of 200.

6.4 Activations, and six layers that never learn



Left: the sigmoid and the ReLU. Right: their slopes. Look at the dotted line: the sigmoid's slope is never above 1/4; the ReLU's is 1 wherever the unit is active.

Section 4.4's backward sweep multiplies the blame by the activation's slope at every layer. With sigmoids that slope is at most 0.25, so after six sigmoid layers the blame reaching the first layer is at most $0.25^6 = 1/4096$ of what left the output: too small to move anything. This is the **vanishing gradient**.



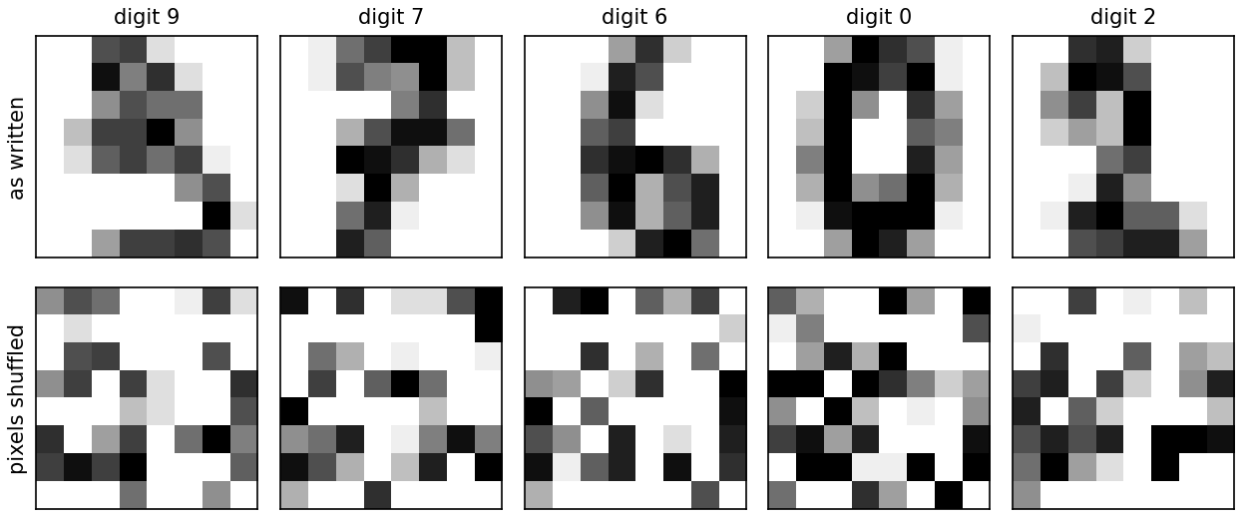
Six hidden layers of 32, identical but for the activation, plain SGD at learning rate 0.1. Look at the sigmoid's line: it sits on 0.1, guessing on ten classes, for all 80 epochs.

The sigmoid network ends at **0.1000**; the same network with ReLU ends at **0.9611**. This is why

deep networks use the ReLU, and lesson 10 meets the problem again in deeper networks.

6.5 Shuffle the pixels: nothing notices

Take one fixed reordering of the 64 pixels and apply it to every image.



Five digits as written, and the same five with their pixels shuffled. Look at the bottom row: to a person they are noise.

The one-hidden-layer network of section 5 scores 0.9602 on the digits as written and 0.9630 on the shuffled ones, three runs each, a difference well inside the run-to-run spread. A dense layer gives every pixel its own weight and never uses which pixel is next to which, so a consistent shuffle changes nothing it can see. For a photograph, where neighbouring pixels are the point, that is a strange blindness: **permute the pixels consistently and nothing here notices.** Lesson 10 is about networks that do notice.

Summary

- **One neuron is one line.** One unit scores 0.5507 on the acceptance test, barely above always saying "accepted", and 0.5000 on the drift problem.
- **A hidden layer moves the data** into coordinates where a line is enough. Two ReLU units written by hand solve the drift problem, 0.9938; each hidden unit is a line, and the output unit votes over them.
- **Training finds such weights more reliably as the layer widens**: on the drift problem 6 runs of 20 succeeded with two units, 17 with four, all 20 with eight; eight units reached 0.9445 on the acceptance test, against a ceiling of 0.97.
- **Learning is a loop**: predict a mini-batch, measure the loss, send the blame back with backpropagation, move each weight against its blame. For one unit the blame is $\hat{y} - y$ times the input.
- **The learning rate** has a narrow useful band: 0.001 still learning after 60 epochs, 2.0 never settling.
- **39 points against 2.8**: a hidden layer is worth a great deal when inputs act in combination and little when they act independently (2.8 points on one split of the digits, 4.7 on another).

Fit the linear model first.

- **In practice**: a large network memorises a small training set, which the validation loss shows first; six sigmoid layers never leave chance, the same with ReLU reaches 0.9611; shuffling the pixels changes nothing for a dense network.

Homework

`Exercises/09_neural_networks.md`, discussed at the start of Lesson 10, **Friday 18 December 2026**.

Notation used in this lesson

Symbol	Meaning
x, X	one input, the matrix of inputs (one example per row)
w, b	weights, bias
z	a unit's weighted sum, $w^\top x + b$
σ	the sigmoid, $1/(1+e^{-z})$
g	a hidden layer's activation (here ReLU, $\max(0, z)$)
$W^{[l]}$, $b^{[l]}$, $Z^{[l]}$, $A^{[l]}$	weights, biases, weighted sums and activations of layer l
$\hat{y}$, y	prediction, true label
L	the loss on one example (cross-entropy)
α	the learning rate
m	number of examples

Further reading

Resource	Type	Why read it
scikit-learn: neural network models	Official docs	`MLPClassifier`, its parameters and its limits, stated plainly
Keras: getting started	Official docs	The `Sequential` model, `compile` and `fit`, with more examples
TensorFlow Playground	Interactive	Add hidden units and watch the boundary change, in the browser
Nielsen, *Neural Networks and Deep Learning*, ch. 1–2	Book, free online	A gentle, picture-led account of networks and backpropagation
`Advanced/09_neural_networks_advanced/` in this repository		Backpropagation in matrix form, gradient checking, initialisation, optimisers: the material this lesson leaves out